

Code Protection

or

How to Get Their Dirty Hands Out Of Our Precious Codebase

- Legal
- Minification
- Obfuscation
- Self-protection

Legal

- Closed licenses
- Lawyers' army
- etc

Минификация

Minification

- Main goal – lower artifact size
- Mostly for “interpreted” languages
- *Could be used as an obfuscation measure*

Minification | Scripts

```
function checkPassword(){  
    var pwd = $('#password').text();  
    if (pwd == "correct") {  
        document.location = "/profile?login=successful"  
    } else {  
        alert("wrong password")  
    }  
    $('#password').text("")  
}
```



Uglify.js

```
checkPassword() =>  
{ "correct"==$('#password').text("")?document.location="/profile?login=successful":(alert("wrong  
password"))}
```

Minification | Scripts



```
1  /*
2  MIT License (c) copyright 2011-2013 original author or authors */
3  window._bvt||(window._bvt=!0,function(r,D){function jb(a){function
4  a[da]("data-node-uid"))&&delete fa[uid]}function l(a){try{return tr
5  function(b){if(!b[y]||b[y]&&!b[y][y]){var d=b.cloneNode(!0);f.$&&f.
6  a.length;for(var b=0;b<a.length;b++){this[b]=a[b]}}function P(a){ret
7  B=a,V=B.document,z=V.documentElement,y="parentNode",G=null,q="/^check
8  x="/^checked|selected$/ ,v=/msie/i.test(navigator.userAgent),fa={},Va
9  "MozTransform","OTransform","msTransform","Transform"],k;for(k=0;k<
10 b){var c=100;try{c=a.filters["DXImageTransform.Microsoft.Alpha"].op
11 this[0]:[]}},last:function(){return A(this.length?this[this.length-
12 X(b.className+" "+c)}}},removeClass:function(a){return this.each(f
13 function(a){b.appendChild(a)}}}),prepend:function(a){return this.e
14 b[a];return b||0},function(a){return a}},before:function(a){return
15 return this.each(function(b){b.parentNode.replaceChild(A.create(a)[
16 return this.each(function(a,b,c){for(var d in e)e.hasOwnProperty(d)
17 "px");null!=b&&(c.style.top=b-f.top+h[1]+"px")));if(!this[0])return
18 var c=a.offsetWidth,a=a.offsetHeight;b&&this.first().css(b);return{
```

Minification

- Main goal – lower artifact size
- Mostly for “interpreted” languages
- *Could be used as an obfuscation measure*

Minification | Binary

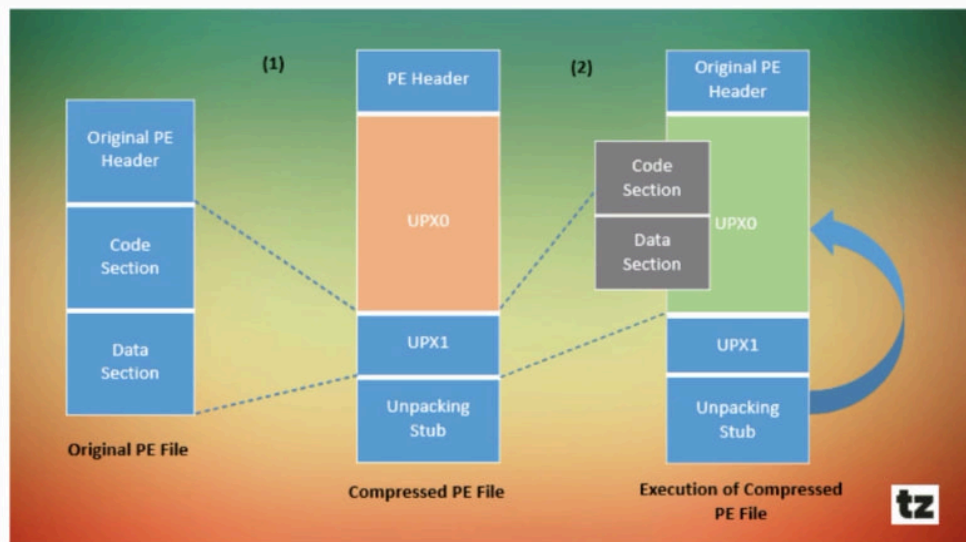
```
int main() {  
    char input[32];  
    fgets(input, 32, stdin);  
    xor(input, KEY)  
    if (!strcmp(input, PASSWORD)) {  
        return 1;  
    }  
    // ...  
}
```



\$ strip --strip-all ./binary

```
int sub4FE7382() {  
    char var28[32];  
    sub4FFBA23(var28, 32, adata23ACeDF);  
    sub4FA7122(var28, adata7F4Ce)  
    if (!sub4FA7982(var28, adata893AEB3)) {  
        return 1;  
    }  
    // ...  
}
```

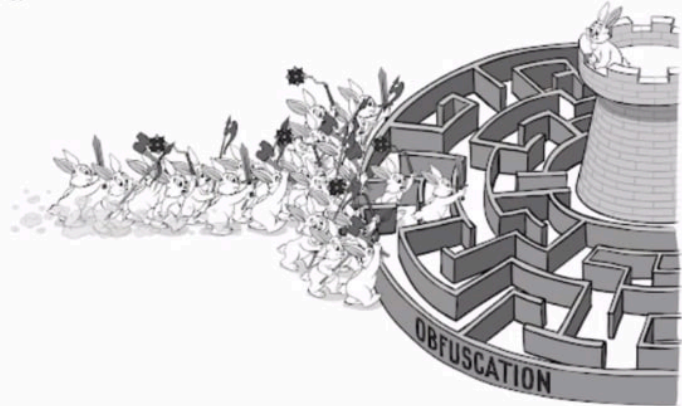
Minification | Compression



Обфускация

Obfuscation

- Main goal – make code analysis harder
- Really wide research area



Constant Expression

Obfuscation | Constant Expressions

```
char input[32];  
fgets(input, 32, stdin);  
if (strlen(input) != 16) {  
    die("wrong")  
}
```

16 => expression

```
char input[32];  
fgets(input, 32, stdin);  
if (strlen(input) != compute(input, GLOBAL_KEY, 0x42, 1337)) {  
    die("wrong")  
}
```

Запутать компилятор/декомпилятор можно:

1. усложнение выражения, приближение к выражению (ряд тейлора, вычисление, циклы, т.д.)

Любую константу можно выразить в ряд тейлора или полинома, также любую математическую функцию (синус)

2. глобальный state, глобальная память на чтение/запись. Компилятор говорит, что это он не может раскурить, лучше оставлю пользователю. Хотите сложнее: используйте глобальные переменные
3. помимо глобальных переменных используйте пользовательский ввод, это еще хуже чем глобальные переменные. Пользовательский ввод надо доказывать в независимости вычисления (smt-решатель)

Константные выражения становятся сильнее если опираются на пользовательский ввод или на глобальные выражения. Это не идеальная защита, но полезная вещь. Минусы данного подхода: это дорого по вычислительным ресурсам

False Branches

Obfuscation | False Branches

```
char input[32];
fgets(input, 32, stdin);
if (strlen(input) != 16) {
    die("wrong")
}
```



No way to know check16 is target

```
char input[32];
fgets(input, 32, stdin);
if (strlen(input) == 16 ) {
    check16(input, KEY16)
} else {
    checkNot16(input, KEYnot16)
}
```

Это те участки кода, куда при обычной работе программы никто не пойдет. Сила метода хороша настолько, насколько вы хорошо опишите ложный branch. Задача состоит в том, чтобы сделать branches более похожими. Иногда может использоваться одна и та же функция проверки, но меняться ключ в зависимости от длины ключа или вычисляться по другому. Чем больше мы похожести добиваемся, тем лучше. В ложные инпуты должно заходить равновероятно

Стойкость:

Работает хорошо, когда

Работает плохо, если

--

Производительность: не сильно жертвуем производительностью, потому рассматриваем положительный исход, который и был прописан изначально. Теряем на компиляции (на этапе сборки), выполняется дольше. Теряем в производительности (спекулятивное исполнение - процессор предсказывает какой бранч будет полезен)

Code Flattening

Obfuscation | Code Flattening

```
char input[32];
fgets(input, 32, stdin);
if (strlen(input) != 16) {
    die("wrong")
}
```

- Code is reordered
- Conditions **flattened**



```
char input[32] = {0}; int len = 0;
char step = 42;
while (1) {
    switch step{
    case 1:
        die("wrong"); return;
    case 2:
        len = strlen(input)
        step = 7; break;
    // ...
    case 7:
        step = (len == 16) ? 13 : 1; break;
    // ...
    case 42:
        fgets(input, 32, stdin);
        step = 2; break;
    }
}
```

Еще более каноничный.

Берем код, делим его на независимые участки кода, дробим детально и упорядочиваем его, потом чтобы вернуть его оригинальный порядок вводим switch case.

Приводит к огромному ветвлению графа, он будет короче, но в разы шире. И вместо того, чтобы анализировать линейно, будем играть в ширину. Это сильно усложняет анализ, также и машинный (но немного, можно получить обратно структурный код)

полезность повышается в случае, если

1. запутаннее код: цикл + вложенный цикл, рекурсия
2. фиктивные записи, мы в них не можем попасть
3. если запутаем переходы, если переходы будут от чего-то зависеть

```
step = (len == 16) ? 13 : 1;
```

2 предыдущих метода круто ложатся в текущую запутанность + делаем кейсы, чтобы были похожи на наш основной код + переходы между ними были тоже похожи + пытаемся запутать логику переходов при помощи конст. выражений

Защита от динамики:

4) отлов неиспользуемых

5) в дебагере можно походить и посмотреть какие кейсы используются.

Работает лучше чем false branches

Хорошая защита от статики

Virtualization

Obfuscation | Virtualization

```
char input[32];  
fgets(input, 32, stdin);  
if (strlen(input) != 16) {  
    die("wrong")  
}
```



Any kind of custom interpreter inside

```
char bytecode[] = "\xde12\xca\x45asdasxz..."  
vm_t *vm = init_vm();  
int code = vm->run(bytecode);  
vm->exit(code);
```

Метод виртуализации.

Байткод - байткод виртуальной машины. Есть процессор, который умеет вычислять наш код. Ничто не мешает нам использовать концепцию вычислителя внутри машины, и превратить наш код с настоящей логикой в код для этой вирт машины. Берем наш код и компилируем его в байткод(питон). Взяли питоновский байткод, записали внутрь нашего бинарника и рядом с ним записали еще интерпритатор питона. Теперь аналитик должен дойти до этого места и понять, что дальше у тебя идет интерпритатор питона, его реверсить не надо, его надо найти, и реверсить пит-байткод питона. Скорее всего интерпритатор будет светиться перед большими вызовами, можно даже на глазоке понять.

Пример:

Польская инверсная запись - виртуальная машина

Минусы:

Замедление, чем больше слоев - тем медленнее, НАМНОГО медленнее

Поддерживать - тяжело и сложно.

Как защищаться от динамического анализа/вмешательства

(Runtime) Self-Protection

- Harden application in a runtime
- Make it harder to attack (malfunction) an app



Нужны методы динамической защиты, от трейсинга, от внедрения в память, отладки, защищать от модификации

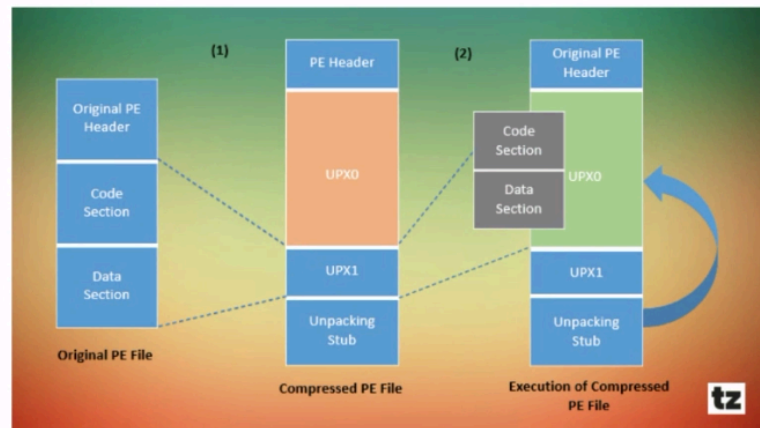
Protection | Integrity Control

- Signatures
 - *Who checks?*
- Self-checks of code hash
 - *Hash value could be patched*
- Remote hash checks
 - *Checking code could be patched*
- Network of overlapping hash checkers

Ответ: Контроль целостности

- Подписи
 - Самопроверка хэш кода
 - Удаленная проверка хэша
 - Проверка цепочки хэша по интернету
- А еще

Protection | ~~Compression~~ Encryption



Protection | Anti-debugging

- *self-ptrace*
- *seccomp*
- *SELinux / kernel params*

Protection | Code Loading

```
var pwd = $("#password").text()
if (!check(pwd)) {
  alert("wrong");
}
// ...
```



Mission-critical code stored somewhere outside

```
var pwd = $("#password").text()
$.get("https://my.server/checkFunc", (code) => {
  eval(code)(pwd);
})
```

Protection | Sentinel Driver

- App
 - Bundled with a self-protection driver
 - Works as usual
- Driver
 - Runs in kernel mode
 - Can't be unloaded
 - Restricts any unauthorized access to app's process or data

Итого

Для защиты используют:

- Legal
- Minification
- Obfuscation
- Self-protection